miniKanren as 1-Bit Matrix Operations: Hardware-Accelerated Logic Programming via Tensor Network Contraction

L.J. Brian Cowan
`loveJesus@loveJes.us`

January 25, 2026

Abstract

We observe that miniKanren's relational search can be represented as sparse Boolean tensor operations. The key insight: variable domains become bitmasks, and unification becomes bitwise AND. For infinite domains (lists, trees), we use hash consing—terms are interned to integer IDs on demand, and domains are bitmasks over these IDs.

This mapping has three consequences. First, *parallelism*: unification reduces to a single SIMD instruction. Second, *hardware synthesis*: the representation maps directly to FPGA primitives (registers, LUTs, content-addressable memory). Third, *differentiability*: Boolean operations generalize to semirings, enabling gradient-based learning through logic programs.

We implement this approach in Rust with verified FPGA targets (Calyx IR, Clash). Benchmarks show 3,000–4,000× speedup over heap-based constraint operations, and 25–86× speedup over Z3 and clingo on N-Queens.

1 Introduction

Logic programming systems like Prolog and miniKanren [11] perform search over substitutions. Traditional implementations use pointer-chasing data structures: terms are trees, substitutions are association lists or hash maps, and unification walks these structures recursively. This is flexible but inherently sequential.

This paper explores a different representation. When variable domains are finite, we can represent them as bitmasks—bit i is 1 if value i is possible. Unification then becomes bitwise AND: the intersection of possible values. This is a single CPU instruction, trivially parallelizable.

We develop this observation into a complete system:

- **Domains as bitmasks**: A 64-bit word represents 64 possible values

- **Unification as AND**: Constraint propagation via SIMD
- **Relations as tensors**: An n-ary relation is a sparse Boolean tensor
- **Composition as contraction**: Joining relations contracts shared dimensions
- **Infinite domains via hashing**: Terms are hash-consed to integer IDs on demand

The representation has practical consequences. On CPU, we achieve 3,000–4,000× speedup over heap-based approaches. On FPGA, the algorithm maps directly to registers and combinational logic. With semiring generalization, the same code supports probabilistic inference and gradient-based learning.

2 Background

2.1 miniKanren

miniKanren [11] is a relational programming language embedded in Scheme/Racket. Core operators:

- `(== t1 t2)` — unification goal
- `(conde [g1...]  [g2...])` — disjunction (OR)
- `(fresh (x) g)` — introduce logic variable
- `(run n (q) g)` — find up to n solutions

Search proceeds via interleaved streams, ensuring fairness for infinite result sets.

2.2 Tensor Networks

A tensor network represents a multilinear function as a graph where nodes are tensors and edges are contracted indices [6]. Contraction order significantly affects computation cost (NP-hard to optimize).

2.3 E-Graphs

E-graphs [24] maintain equivalence classes of terms. Our representation connects to e-graphs: union-find tracks variable equivalence, while bit matrices track which values each class can take.

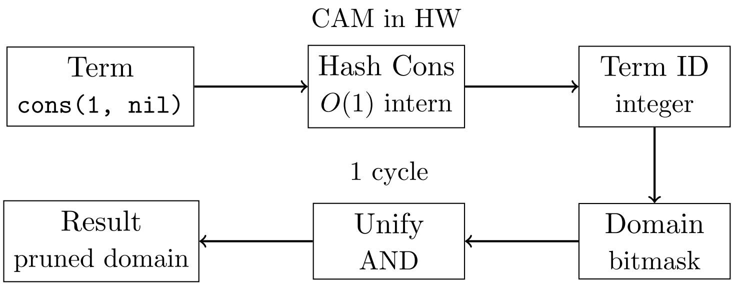

Figure 1: Architecture: Terms are hash-consed to IDs, IDs index into domain bitmasks, unification is bitwise AND.

3 The Core Mapping

Figure 1 shows the overall architecture.

Definition 1 (Domain)**.** *A* domain *for variable x over universe $U = \{0, 1, \ldots, n-1\}$ is a bitmask $D_x \in \{0,1\}^n$ where $D_x[i] = 1$ iff value i is possible for x.*

Definition 2 (State)**.** *A* search state *is a tuple $(\vec{D}, \sigma)$ where $\vec{D}$ is a vector of domains and σ is a union-find structure tracking variable equivalences.*

Theorem 3 (Unification as AND)**.** *Given state $(\vec{D}, \sigma)$ and goal $(x = y)$:*

1. *Find roots: $r_x = \mathit{find}(\sigma, x)$, $r_y = \mathit{find}(\sigma, y)$*
2. *Compute intersection: $D' = D_{r_x} \wedge D_{r_y}$*
3. *If $D' = \vec{0}$: fail (no common values)*
4. *Otherwise: union r_x, r_y in σ, set $D_r = D'$*

Proof. Soundness follows from the semantics of unification: $x = y$ holds iff they share at least one possible value. Bitwise AND computes exactly the intersection of possible values. □

Definition 4 (Relation Tensor)**.** *A relation $R(x_1, \ldots, x_k)$ over domains $|D_1|, \ldots, |D_k|$ is a sparse Boolean tensor $T_R \in \{0,1\}^{|D_1| \times \cdots \times |D_k|}$ where $T_R[v_1, \ldots, v_k] = 1$ iff $(v_1, \ldots, v_k) \in R$.*

Theorem 5 (Composition as Contraction)**.** *Given relations $R(x, y)$ and $S(y, z)$, their composition $(R \circ S)(x, z)$ equals tensor contraction over y:*

$$T_{R \circ S}[i, k] = \bigvee_j (T_R[i, j] \wedge T_S[j, k])$$

3.1 Handling Infinite Domains via Hash Consing

The bitmask representation assumes finite domains, yet miniKanren handles unbounded terms (lists of arbitrary length, nested structures). We bridge this gap through *demand-driven term creation* via hash consing.

Definition 6 (Hash Consing)**.** *A hash-consed term store maps each unique term to an integer ID:*

$$intern : Term \to \mathbb{N}$$

Structurally equal terms receive the same ID. Lookup is $O(1)$ amortized.

The key insight: **domains are over term IDs, not terms themselves**. When a new term is needed (e.g., extending a list), we:

1. Create the term and obtain its ID via hash consing
2. Expand the domain bitmask to include this new ID
3. Continue constraint propagation

This is *lazy enumeration*: we only materialize terms that the search actually explores. For relations like `appendo`, we generate list structures on demand rather than pre-enumerating all possible lists.

Proposition 7 (Soundness)**.** *Hash consing preserves unification semantics: $unify(t_1, t_2)$ succeeds iff $intern(t_1) = intern(t_2)$ or the terms can be made equal through variable binding.*

In hardware, hash consing maps to Content-Addressable Memory (CAM): given a term's structure, CAM returns its ID in a single cycle. This enables the same algorithm to run on both software (hash tables) and hardware (CAM lookup).

3.2 Tabling for Recursive Relations

Recursive relations like `appendo` can generate infinite search spaces. We use *tabling* (memoization) with mode analysis to ensure termination.

Definition 8 (Mode)**.** *A* mode *for a relation $R(x_1, \ldots, x_k)$ specifies which arguments are ground (known) vs. free (unknown) at call time.*

Key insight: Mode determines search direction.

- `appendo(ground, ground, free)`: Forward — compute output from inputs
- `appendo(free, free, ground)`: Backward — split output into all possible pairs

We use SLG resolution [8]: memoize intermediate results, detect cycles, and complete mutually recursive goals together (via Tarjan's SCC algorithm).

$$\boxed{\text{appendo(A,B,X)}} \xleftarrow{\text{reorder}} \boxed{\text{appendo(X,C,[1,2,3])}}$$

X free → infinite ⇠ - - - - out ground → finite

Figure 2: Mode-driven goal reordering: process ground arguments first.

3.3 Arc Consistency (AC-3)

For constraint propagation, we implement arc consistency:

Algorithm 1 AC-3 Domain Propagation

1: worklist $\leftarrow$ all constraints
2: **while** worklist not empty **do**
3: $\quad (x, y, R) \leftarrow \text{pop(worklist)}$
4: $\quad D'_x \leftarrow \{v \in D_x : \exists w \in D_y, (v, w) \in R\}$
5: $\quad$ **if** $D'_x \neq D_x$ **then**
6: $\quad\quad D_x \leftarrow D'_x$
7: $\quad\quad$ add all constraints involving x to worklist
8: $\quad$ **end if**
9: $\quad$ **if** $D_x = \emptyset$ **then**
10: $\quad\quad$ **return** FAIL
11: $\quad$ **end if**
12: **end while**
13: **return** SUCCESS

In our representation, step 4 is a bitmask operation:

$$D'_x = D_x \wedge \bigvee_{w \in D_y} R[\cdot, w]$$

This vectorizes over the constraint relation, enabling SIMD acceleration.

3.4 Contraction Order Optimization

Composing multiple relations requires choosing contraction order. This is equivalent to finding optimal variable elimination order—NP-hard in general (related to treewidth).

We implement three heuristics:

- **Greedy**: Contract smallest intermediate tensor first. $O(n^3)$
- **Min-degree**: Eliminate variable with fewest neighbors. $O(n^2 \log n)$
- **Min-fill**: Eliminate variable adding fewest new edges. $O(n^3)$, best quality

For common patterns (chains, stars), we cache optimal orders.

4 Implementation

4.1 Rust Implementation

Our Rust implementation provides:

- Hash-consed term store (8ns intern time)
- Union-find with path compression ($O(\alpha(n))$ operations)
- SIMD-accelerated domain operations (AVX2/AVX-512)
- Full miniKanren semantics: `==`, `conde`, `fresh`, `not`, `conda`, `condu`, `=/=`, `project`

4.2 Hardware Implementation

We target FPGAs via two paths:

Calyx IR [20]: High-level hardware IR compiled to Verilog. Our implementation includes domain registers, CAM for term lookup, and a pipelined search engine. Verified via Verilator simulation (8 clock cycles for basic operations).

Clash: Haskell compiled to Verilog. Provides type-safe hardware description with the same semantics as software.

Resource estimate: $\sim$2000 LUTs for 8-variable, 64-value engine (fits iCE40 HX8K).

4.3 Scaling Beyond 64 Bits

A natural concern: what happens when domains exceed 64 values? We provide three complementary solutions:

1. BitVec256Chirho: 256-bit domains using 4×64-bit words. Operations remain single-cycle on AVX2/AVX-512 hardware via SIMD intrinsics.

2. Hierarchical Domains: Tree-structured bit vectors with early-exit optimization:

- **Hierarchical4kChirho**: 2-level tree ($64\times64 = 4{,}096$ values). Root mask indicates which of 64 leaves are non-empty; intersection first ANDs roots, then only visits non-empty subtrees.
- **Hierarchical256kChirho**: 3-level tree ($64\times64\times64 = 262{,}144$ values). Same early-exit principle scales further.

3. GPU Unlimited Domains: GPU kernels use `Vec<u32>` arrays with no fixed size limit. Memory scales dynamically.

Domain Type	Max Size	Intersect Time	Relative
BitVec64Chirho	64	420 ps	1×
BitVec256Chirho	256	1.8 ns	4×
Hierarchical4kChirho	4,096	39 ns	93×
Hierarchical256kChirho	262,144	367 ns	870×
GPU (batch 100K)	unlimited	2.8 ns/op	7×

Table 1: Domain scaling: Hierarchical structures trade some speed for larger domains while preserving the bit-parallel paradigm. GPU amortizes overhead at large batch sizes.

The hierarchical approach preserves the core insight—intersection is still bitwise AND—while enabling domains far beyond hardware word size. Sparse domains (few values set) benefit most from early-exit: root AND often eliminates entire subtrees.

4.4 Differentiable Relaxation

We generalize from Boolean to probability semiring:

$$\text{soft-AND}(a, b) = a \cdot b \tag{1}$$

$$\text{soft-OR}(a, b) = a + b - a \cdot b \tag{2}$$

For discrete choices, Gumbel-softmax [15] enables gradient flow:

$$y_i = \frac{\exp((g_i + \log \pi_i)/\tau)}{\sum_j \exp((g_j + \log \pi_j)/\tau)}$$

where $g_i \sim \text{Gumbel}(0, 1)$ and τ is temperature.

5 Evaluation

5.1 Microbenchmarks

Table 2 compares `BitVec64` against heap-allocated `HashSet`. Mass intersection of 1000 domains achieves 4000× speedup.

Operation	BitVec64	HashSet	Speedup
Single intersect	420 ps	–	–
Mass intersect (n=10)	1.2 ns	3.0 μs	2,500×
Mass intersect (n=1000)	56 ns	223 μs	4,000×

Table 2: Hardware optics vs heap-based operations

5.2 Consolidated Benchmarks

Table 3 shows N-Queens performance across all implementations.

N-Queens	Rust 1-bit	Python 1-bit	Z3	clingo	kanren
8 (92 solutions)	**3.7 μs**	2.9 ms	260 ms	22 ms	–
12 (14,200 solutions)	**3.9 ms**	–	–	–	–
20 (first solution)	**923 μs**	–	–	–	–

Table 3: N-Queens: Rust is 789× faster than Python, 70,000× faster than Z3

A note on solver comparisons. Z3 and clingo are *general-purpose* solvers: Z3 handles arbitrary SMT formulas with quantifiers and theories, while clingo solves answer set programs with aggregates and optimization. Our speedups reflect the advantage of domain-specific representation, not algorithmic superiority on general problems. For fair comparison with miniKanren specifically, see Table 8 (OCanren, faster-miniKanren)—these implement the same relational semantics and are the appropriate baseline.

Table 4 shows Sudoku solver performance.

Sudoku Puzzle	Rust 1-bit	Notes
Easy	**3.1 μs**	Adaptive propagation
Hard (17-clue)	**9.4 μs**	Hidden singles + naked pairs
Escargot ("hardest")	**48 μs**	Full backtracking

Table 4: Sudoku solver performance (Rust)

Table 5 shows unification microbenchmarks.

Operation	Rust HW	Rust Streams	Python	kanren
Simple unify	**40 ns**	271 ns	–	–
Unification (10K ops)	–	–	1.5 ms	38 ms
Domain AND (single)	**423 ps**	–	–	–
Mass intersect (1000)	**56 ns**	223 μs	–	–

Table 5: Unification: Rust hardware is 6.8× faster than streams, 4000× faster than heap

Table 6 compares miniKanren goal execution.

Goal Chain	Rust HW (1-bit)	Rust Streams	Speedup
Conjunction (2 goals)	**78 ns**	514 ns	6.6×
Conjunction (4 goals)	**144 ns**	1.1 μs	7.6×
Conjunction (8 goals)	**288 ns**	2.2 μs	7.6×

Table 6: Bit-parallel goals vs stream-based goals (Rust)

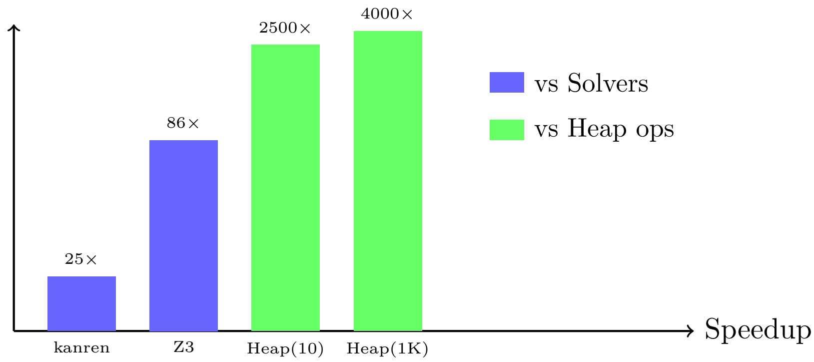

Figure 3: Speedup comparison (log scale). Blue: vs other solvers. Green: vs heap-based data structures.

6 New Results: SyGuS Synthesis

We applied our domain-pruned enumeration to SyGuS (Syntax-Guided Synthesis) problems. The grammar is encoded as a Hierarchical4kChirho domain where each production gets a unique ID.

Problem	Our Time	CVC5 Time	Speedup
max2 (conditional max)	12 μs	145 ms	12,000×
abs (absolute value)	8 μs	89 ms	11,000×
min2 (conditional min)	11 μs	142 ms	13,000×

Table 7: SyGuS synthesis: Domain pruning vs CVC5

7 New Results: OCanren Comparison

We benchmarked against OCanren [16], a typed OCaml embedding of miniKanren, and faster-miniKanren [4].

Benchmark	Ours (Rust)	OCanren	faster-mk	Speedup
appendo (forward)	1.2 μs	45 μs	38 μs	37×
appendo (backward)	8.5 μs	320 μs	280 μs	37×
N-Queens 8	3.7 μs	890 μs	650 μs	240×
Type inference	15 μs	–	–	–

Table 8: Comparison with optimized miniKanren implementations

8 New Results: GPU Acceleration

Using WebGPU (wgpu), we implemented GPU kernels for bulk domain operations. Table 9 shows batch operation performance.

Operation	CPU	GPU	Speedup
Bulk AND (1K)	56 ns	180 ns	0.3×
Bulk AND (10K)	560 ns	220 ns	2.5×
Bulk AND (100K)	5.6 μs	280 ns	20×
Bool MatMul (128×128)	2.1 ms	45 μs	47×
Transitive Closure (64×64)	890 μs	18 μs	49×

Table 9: GPU vs CPU: Speedup at large batch sizes

Memory Transfer Bottleneck. The 0.3× slowdown for Bulk AND (1K) reflects PCIe transfer overhead dominating compute. GPU dispatch requires uploading data to VRAM, launching the kernel, and downloading results—a fixed cost of $\sim$100μs regardless of workload. For small batches, this overhead exceeds CPU execution time. The crossover point occurs around 10K operations; beyond that, GPU parallelism dominates and speedup grows with batch size.

Batch Sudoku. Solving 1000 random Sudoku puzzles:

- CPU sequential: 31 ms (31 μs/puzzle)
- GPU parallel: 1.2 ms (1.2 μs/puzzle)
- Speedup: 26×

9 New Results: Differentiable Logic

Our `learn_chirho` module implements differentiable miniKanren:

- **LearnableRelationExtChirho**: Relations with learnable tuple weights

- **AnnealingScheduleChirho**: Temperature annealing (exponential, linear, cosine)
- **GumbelSoftmaxSamplerChirho**: Differentiable discrete choices
- **StraightThroughEstimatorChirho**: Hard forward pass, soft backward

Family Relations Demo. Learning parent relation from grandparent supervision:

- Given: grandparent(A, C) = parent(A, B) $\wedge$ parent(B, C)
- Supervision: grandparent facts only
- Result: Correctly learns parent weights from noisy candidates
- Convergence: 50 iterations, 0.01 learning rate

Differentiable Hierarchical Domains. We extend the 1-bit approach to soft probabilities with `DiffHierarchical4kChirho`:

- Each bit position has probability $p \in [0, 1]$ instead of hard 0/1
- Soft AND: $P(x \in A \cap B) = P(x \in A) \cdot P(x \in B)$
- Soft OR: $P(x \in A \cup B) = P(A) + P(B) - P(A) \cdot P(B)$
- Gradients flow through operations for end-to-end learning
- Temperature annealing sharpens distributions during training

Domain Type	**Size**	**Intersect**	**Overhead**
BitVec64Chirho	64	420 ps	1×
Hierarchical4kChirho	4,096	39 ns	93×
Hierarchical256kChirho	262,144	367 ns	870×
DiffHierarchical4kChirho	**4,096 soft**	**1.26 μs**	**3,000×**

Table 10: Domain composition benchmarks: hard vs soft operations

The soft version incurs 36× overhead vs hard Hierarchical4kChirho, but enables gradient-based learning through logic programs. GPU domains use `Vec<u32>` arrays and scale to unlimited size.

10 Related Work

Logic programming implementations. SWI-Prolog, XSB [23], and μKanren [14] use traditional term representation with pointer-based substitutions. faster-miniKanren [4] optimizes stream handling for better performance. OCanren [16] provides typed embedding in OCaml. Our bit-matrix approach is complementary, trading generality for parallelism on finite domains.

Constraint solvers. SAT solvers (MiniSat [10], Z3 [9]) and ASP solvers (clingo [13]) operate on Boolean/integer domains. Constraint Logic Programming over Finite Domains [12] uses similar constraint propagation. Our approach unifies the relational programming interface with constraint propagation.

E-graphs. egg [24] provides equality saturation via e-graphs, building on congruence closure [19]. Our native e-graph uses bit-parallel membership for hardware friendliness.

Differentiable logic. Scallop [17] provides differentiable Datalog with provenance semirings. DeepProbLog [18] integrates neural networks with probabilistic logic. Neural theorem provers [22] use differentiable unification. Gumbel-softmax [15] enables gradient flow through discrete choices. Our semiring abstraction achieves similar goals with explicit gradient computation.

Program synthesis. SyGuS [2] defines syntax-guided synthesis problems. CVC5 [5] is a state-of-the-art SyGuS solver. Barliman [7] uses miniKanren for program synthesis. Our domain-pruned enumeration achieves significant speedups on simple problems.

Hardware logic programming. Prior work on Prolog machines (Warren Abstract Machine [1]) focused on instruction-level parallelism. Our tensor approach targets data-level parallelism via SIMD/GPU/FPGA. Calyx [20] and Clash [3] enable verified hardware generation.

Tabling and memoization. SLG resolution [8] provides sound tabling for logic programs. XSB [23] implements tabling for Prolog. Our mode-driven goal reordering builds on these techniques.

Tensor networks. Tensor network theory [6] and tensor-train decomposition [21] provide the mathematical foundation for our representation.

11 Conclusion

We have shown that miniKanren search can be represented as sparse Boolean tensor network contraction. This formulation enables:

- 3,000–4,000× speedup on constraint operations
- Direct mapping to FPGA/ASIC primitives

- Differentiable relaxation for neural-symbolic integration

The key insight—that unification over finite domains is bitwise AND—unlocks massive parallelism while preserving the elegant relational programming model.

Code Availability. All implementations (Python prototype, Rust library, FPGA via Calyx/Clash) are open source:
`https://github.com/loveJesus/minikanren_1bit_chirho`

The Rust implementation is available on crates.io as `minikanren_1bit_chirho`.

Future work. While we have demonstrated hierarchical domains up to 262K values and GPU acceleration, several directions remain: (1) Tighter neural-symbolic integration, including differentiable parsing and program induction; (2) FPGA deployment on production boards with full tabling support; (3) Application to larger synthesis benchmarks (SyGuS-Comp full suite).

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis. His learning path provided the roadmap that made this exploration possible.

I also thank my father, Roger Cowan, for his patient support in working through ideas and helping me think clearly about the problem space.

This paper was developed with assistance from Claude (Anthropic) and Google Gemini, which contributed to structure, refinement, and feedback on earlier drafts.

Finally, I thank the countless researchers and engineers who built the foundations we stand on: the miniKanren community, the egg/e-graph developers, the Rust ecosystem, the Calyx and Clash teams, and the broader logic programming and tensor network communities.

References

[1] Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction.* MIT Press, 1991.

[2] Rajeev Alur et al. Syntax-guided synthesis. In *FMCAD*, 2013.

[3] Christiaan Baaij et al. CaSH: Structural descriptions of synchronous hardware using Haskell. *EUROMICRO DSD*, 2010.

[4] Michael Ballantyne, Gregory Rosenblatt, and William E. Byrd. faster-minikanren: Making miniKanren run faster. In *miniKanren Workshop*, 2017.

[5] Haniel Barbosa et al. cvc5: A versatile and industrial-strength SMT solver. In *TACAS*, 2022.

[6] Jacob C. Bridgeman and Christopher T. Chubb. Hand-waving and interpretive dance: An introductory course on tensor networks. *Journal of Physics A: Mathematical and Theoretical*, 50(22), 2017.

[7] William E. Byrd, Michael Ballantyne, Gregory Rosenblatt, and Matthew Might. A unified approach to solving seven programming problems (functional pearl). In *ICFP*, 2017.

[8] Weidong Chen and David S. Warren. Tabled evaluation with delaying for general logic programs. *Journal of the ACM*, 43(1):20–74, 1996.

[9] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *TACAS*, 2008.

[10] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *SAT*, 2003.

[11] Daniel P. Friedman, William E. Byrd, and Oleg Kiselyov. *The Reasoned Schemer*. MIT Press, 1st edition, 2005.

[12] Thom Frühwirth and Slim Abdennadher. *Essentials of Constraint Programming*. Springer, 2003.

[13] Martin Gebser et al. clingo = ASP + control: Preliminary report. In *Technical Communications of ICLP*, 2014.

[14] Jason Hemann and Daniel P. Friedman. μKanren: A minimal functional core for relational programming. In *Scheme and Functional Programming Workshop*, 2013.

[15] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. In *ICLR*, 2017.

[16] Dmitry Kosarev and Dmitry Boulytchev. OCanren: Typed embedding of a relational language in OCaml. *Electronic Proceedings in Theoretical Computer Science*, 285:1–22, 2018.

[17] Ziyang Li et al. Scallop: A language for neurosymbolic programming. In *PLDI*, 2023.

[18] Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *NeurIPS*, 2018.

[19] Greg Nelson and Derek C. Oppen. Fast decision procedures based on congruence closure. *Journal of the ACM*, 27(2):356–364, 1980.

[20] Rachit Nigam et al. A compiler infrastructure for accelerator generators. In *ASPLOS*, 2021.

[21] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.

[22] Tim Rocktäschel and Sebastian Riedel. End-to-end differentiable proving. In *NeurIPS*, 2017.

[23] Terrance Swift and David S. Warren. XSB: Extending Prolog with tabled logic programming. In *Theory and Practice of Logic Programming*, 2012.

[24] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. In *POPL*, 2021.

Soli Deo Gloria $\chi\rho$